

Fraud Ring Detection Using Heterogeneous Graph Neural Networks

Milind Sahu

BS–MS Student (Year 1)

Indian Institute of Science Education and Research Tirupati

April 2025

Abstract

Financial fraud often involves groups of accounts acting together rather than isolated suspicious transactions. Transaction-level detection methods tend to miss these coordinated patterns because each individual transaction may appear legitimate on its own. This project presents HETGNN-FR, a system that models financial data as a heterogeneous graph with four node types (accounts, devices, merchants, IP addresses) and applies a Heterogeneous Graph Transformer to detect coordinated fraud rings. A Time2Vec encoding layer is included to capture temporal clustering behaviour, since fraud rings tend to operate within short time windows. Training proceeds in two stages: the graph neural network is first trained at the account level to produce discriminative embeddings, then a separate ring-level MLP classifier is trained on top of those embeddings. On a synthetic dataset of 300 accounts with 8 injected fraud rings, the system achieves a mean validation AUC of 0.863 across five random seeds (range 0.65–1.00), with ring-level recall of 1.00 at the 0.50 decision threshold. A basic explainability module and a Streamlit interface enable users to interactively inspect detected rings.

Contents

1	Introduction	3
2	Related Work	4
3	Problem Formulation	6
3.1	Graph Construction	6
3.2	Fraud Ring Definition	6
3.3	Task Definition	6
4	Dataset	6
4.1	Dataset Statistics	7
4.2	Scale Presets	7

4.3	Fraud Ring Injection	7
4.4	Node Features	8
5	Model Architecture	8
5.1	Input Projection	9
5.2	Time2Vec Temporal Encoding	9
5.3	Heterogeneous Graph Transformer Layers	10
5.4	Sliding-Window Ring Extraction	10
5.5	Ring Feature Vector	10
5.6	Ring Classifier	11
6	Training	11
6.1	Stage 1 — Account-Level GNN Training	11
6.2	Stage 2 — Ring Classifier Training	12
7	Results	13
7.1	Baselines	13
7.2	GNN Node Classification	13
7.3	Ring Classification	14
7.4	Ablation: Temporal Encoding and Pooling	14
7.5	Statistical Testing	14
7.6	Scale Experiment	15
7.7	Ring Score Distribution	16
8	Explainability	16
8.1	Shared Infrastructure Detection	16
8.2	Edge Suspicion Scoring	16
8.3	Gradient-Based Feature Importance	17
9	Demo Application	17
10	Discussion	18
10.1	What the Experiments Revealed	18
10.2	Limitations	18
10.3	Future Directions	19
11	Conclusion	19
A	File Structure	23
B	Hyperparameter Summary	25

1 Introduction

HETGNN-FR emerged from a specific failure mode observed when applying standard account-level classifiers to a synthetic financial dataset: eight injected fraud rings, each with 4–10 members sharing a device and IP address and completing circular transfers within a two-hour window, produced individual transactions that were individually unremarkable in amount and timing, yet unmistakable in aggregate. The coordinated pattern is invisible to per-transaction classifiers and only apparent when accounts, devices, and IP addresses are examined as a connected structure.

A fraud ring typically involves several accounts performing circular transfers among members within a short time window to move funds quickly. Modelling accounts, devices, IP addresses, and merchants as nodes and their relationships as edges allows fraud rings to manifest as suspicious subgraph patterns. Graph Neural Networks learn what those patterns look like from labelled examples, eliminating manually designed detection rules.

The proposed system, HETGNN-FR, combines several components into a single end-to-end pipeline:

- A heterogeneous graph with four node types and four edge types, built from transaction records
- A Time2Vec layer that encodes edge timestamps into learnable periodic and linear components
- Two HGTCConv layers from PyTorch Geometric that produce per-account embeddings
- A sliding time-window extractor that generates candidate fraud ring subgraphs
- A three-layer MLP ring classifier trained on pooled embeddings and structural features
- An explainability module based on shared infrastructure detection and edge suspicion scoring
- A Streamlit demo that loads the trained checkpoint and displays rings sorted by predicted fraud probability

Three design decisions in this project deserve specific motivation. First, the sliding window for ring extraction is set to 6 hours with a 2-hour stride. Windows of 4 hours missed rings whose transactions spanned slightly longer durations due to timestamp jitter introduced in the data generator; windows of 12 hours introduced enough spurious connected components to raise the false-positive rate noticeably. The 6-hour width was the narrowest setting that recovered all 8 injected rings across all 5 evaluation seeds. Second, a two-layer MLP projection is appended after the second HGTCConv layer. Without this projection, account embeddings from different fraud and legitimate clusters overlapped substantially in the downstream ring classifier, reducing ring-level validation AUC by approximately 0.03–0.05. Third, the ring pooling uses three operations in parallel — mean, max, and hub-node (highest-degree account in the ring) — rather than mean alone. The hub node captures the central coordinator account that appears in more transactions than other members, which

is a consistent structural property of the injected rings.

The rest of the report covers related work, the problem formulation, the dataset, model architecture, training procedure, results, explainability, the demo, and a discussion of limitations and future directions.

2 Related Work

Logistic regression, random forests, and gradient boosting perform well when fraud is carried out by isolated actors on tabular transaction features, but produce no account-relationship representations and cannot detect coordinated fraud where each transaction appears normal and the suspicious signal is distributed across a group.

Graph-based fraud detection attracted growing attention from around 2018. Liu et al. (2018) [8] applied label propagation on a transaction graph for malicious account detection. GraphSAGE [1] and Graph Attention Networks [2] were later adapted for fraud by treating transactions or accounts as nodes in a homogeneous graph. These methods do not natively support multiple node or edge types, so device-sharing edges and money-transfer edges are collapsed into a single relation, losing the distinguishing signal that makes ring detection tractable.

The Heterogeneous Graph Transformer (HGT) [3] by Hu et al. (2020) introduces type-specific attention mechanisms, making it suitable for financial graphs where a device-sharing relationship carries different information from a money transfer.

Industrial-Scale Heterogeneous GNN Fraud Detection

Rao et al. (2021) [9] propose xFraud, a heterogeneous GNN deployed at Alipay (Ant Group) on graphs with hundreds of millions of nodes. xFraud combines a relational graph attention network with an explainability head that attributes predictions to specific edge types. Unlike xFraud, which requires a continuously maintained production graph with real-time feature pipelines and is evaluated on fully proprietary data, the present work targets a self-contained research prototype where the end-to-end pipeline — including data generation, training, and interactive inspection — can be reproduced from a single script.

Zhang et al. (2022) [10] introduce PICK-HARGE, a hierarchical graph neural network for fraud detection in e-commerce networks at JD Finance, operating over multi-relational graphs of orders, devices, and users. PICK-HARGE employs a two-level aggregation scheme that separately models within-ring and cross-ring relationships before a final fraud scoring layer. Unlike PICK-HARGE, which presupposes that ring membership is known at training time and uses it to construct the hierarchy, HETGNN-FR must first *discover* candidate rings via sliding-window extraction before classifying them, making it applicable to settings where ring boundaries are not annotated.

Temporal Graph Networks for Financial Fraud

Pareja et al. (2020) [11] and Rossi et al. (2020) [7] introduce temporal graph learning methods that process edges as a continuous stream rather than a static snapshot. Deng et al. (2023) [12] apply TGN-style memory modules to financial transaction graphs, demonstrating that continuous-time representations outperform static GNNs on anomaly detection benchmarks with dense temporal dependencies. Unlike these approaches, which require maintaining a memory state per node that must be updated at each transaction event, HETGNN-FR uses a simpler Time2Vec pre-encoding of timestamps that adds a fixed overhead of 16 learnable parameters per transaction edge and integrates directly into a standard HGTCConv forward pass without custom CUDA kernels or streaming infrastructure.

Class-Imbalance-Aware Fraud Detection

Liu et al. (2021) [13] propose PC-GNN (Pick and Choose GNN), which addresses the severe class imbalance in fraud detection by constructing over- and under-sampled neighbourhood subgraphs per class during training. Huang et al. (2022) [14] introduce FRAUDRE, which adds a fraud-aware aggregation layer that down-weights legitimate neighbours during message passing to prevent them from diluting the fraud signal. Unlike PC-GNN and FRAUDRE, which operate at the node (account) level and do not produce group-level predictions, HETGNN-FR addresses imbalance at two levels: inverse-frequency class weighting during GNN pre-training, and again during ring-classifier training where fraud rings constitute only 5–15% of candidates.

Explainability in Fraud GNNs

Explainability in GNNs is commonly addressed with GNNExplainer [6], which identifies subgraphs and features relevant to a prediction. HETGNN-FR uses a simpler gradient-based saliency approach alongside structural pattern matching (shared device and IP detection), which is more directly interpretable for a fraud analyst who needs to understand which shared infrastructure triggered the flag rather than which abstract subgraph pattern the model attended to.

For temporal modelling, JODIE [5] and other temporal GNN methods track user behaviour over time using continuous-time representations. HETGNN-FR uses Time2Vec [4], which encodes timestamps into fixed-size vectors using learnable linear and sinusoidal components. This is sufficient for the temporal clustering behaviour observed in fraud rings and requires no recurrent state.

3 Problem Formulation

3.1 Graph Construction

The financial ecosystem is modelled as a heterogeneous graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$. The node set has four types:

$$\mathcal{V} = \mathcal{V}_{\text{account}} \cup \mathcal{V}_{\text{device}} \cup \mathcal{V}_{\text{merchant}} \cup \mathcal{V}_{\text{ip}}$$

The edge set has four relationship types:

$$\begin{aligned} \mathcal{E}_{\text{transacts}} &= \{(a_i, a_j) \mid \text{account } a_i \text{ transferred funds to } a_j\} \\ \mathcal{E}_{\text{uses_device}} &= \{(a_i, d_k) \mid \text{account } a_i \text{ was accessed from device } d_k\} \\ \mathcal{E}_{\text{accesses_ip}} &= \{(a_i, p_k) \mid \text{account } a_i \text{ connected from IP address } p_k\} \\ \mathcal{E}_{\text{sells_to}} &= \{(a_i, m_k) \mid \text{account } a_i \text{ transacted with merchant } m_k\} \end{aligned}$$

Transaction edges are timestamped and carry a dollar amount as edge attributes. The remaining three edge types are static relationships.

3.2 Fraud Ring Definition

A fraud ring R is a set of accounts $\{a_1, a_2, \dots, a_k\}$ satisfying at least one of the following:

- **Shared infrastructure:** All members share a common device ID or IP address
- **Circular transfers:** A transaction cycle $a_1 \rightarrow a_2 \rightarrow \dots \rightarrow a_k \rightarrow a_1$ exists
- **Temporal clustering:** All intra-ring transactions occur within a two-hour window

The synthetic dataset generator introduces realistic noise into all three criteria: only 70% of ring members adopt the shared device (`device_share_rate`), only 65% adopt the shared IP (`ip_share_rate`), 80% of circular transfer edges are retained (`cycle_completeness`), and timestamps carry ± 45 -minute uniform jitter. These parameters are discussed further in Section 4.

3.3 Task Definition

Given $\mathcal{G}$, the system must:

1. Learn embeddings $\mathbf{h}_a \in \mathbb{R}^d$ for each account $a \in \mathcal{V}_{\text{account}}$
2. Extract candidate ring subgraphs $\{R_1, \dots, R_m\}$ using a sliding time window
3. Classify each candidate ring R_i as fraudulent or legitimate

4 Dataset

The v5 dataset generator introduces five noise parameters absent from the original version, chosen to make the synthetic rings harder to detect than perfectly clean injected examples:

`device_share_rate` (default 0.70), `ip_share_rate` (default 0.65), `cycle_completeness` (default 0.80), `time_jitter_minutes` (default 45), and `ring_amount_noise` (default 0.30). At these default settings, roughly 30% of ring members do not share the canonical ring device, 20% of circular transfer edges are dropped, and timestamps deviate by up to 45 minutes from the ideal two-hour window. Real-world fraud datasets are proprietary, so a synthetic dataset was generated; the noise parameters are designed to close the gap between synthetic and realistic rings.

The generator also supports three scale presets — `small`, `medium`, and `large` — used in the scalability experiments in Section 7.6.

4.1 Dataset Statistics

Table 1: Synthetic Dataset Overview (small scale, default noise parameters)

Entity	Count	Notes
Accounts	300	Credit score, average balance, account age
Devices	120	Mobile, desktop, tablet
Merchants	40	Retail, food, travel, gaming, crypto
IP Addresses	80	US, CN, RU, DE, BR
Legit Transactions	1 200	Exponential distribution, mean \$200
Fraud Transactions	≈ 80	Uniform \$500–\$5 000 $\times$ amount noise
Injected Fraud Rings	8	4–10 accounts each

4.2 Scale Presets

Table 2: Dataset Scale Presets

Scale	Accounts	Devices	Merchants	IPs	Fraud Rings
small	300	120	40	80	8
medium	1 500	500	150	350	30
large	6 000	2 000	500	1 200	100

4.3 Fraud Ring Injection

For each of the eight injected rings, the generator selects a random group of 4–10 accounts and injects fraud with the noise parameters above. Because `device_share_rate` = 0.70, approximately two members per ring use a random device rather than the shared one, which means shared-device detection alone is no longer sufficient to flag every ring. Ring labelling for the ring classifier uses Jaccard overlap against ground-truth ring sets with a 0.5 IoU threshold, so partial rings captured by the sliding window can still receive a positive label.

Approximately 7–10% of accounts are involved in fraud, which is a realistic class imbalance ratio for this setting.

4.4 Node Features

Table 3: Features per Node Type

Node Type	Features	Dimension
Account	Credit score, average balance, account age	3
Device	Encoded device category	1
Merchant	One-hot category embedding	5
IP	One-hot country embedding	5

Transaction edges carry a two-dimensional attribute vector: normalised timestamp and transaction amount.

5 Model Architecture

The HETGNN-FR pipeline has three stages: a HGT encoder with 64-dimensional account embeddings, a sliding-window ring extractor with two-pass deduplication, and a 197-dimensional MLP ring classifier. Figure 1 illustrates the full system. The v5 codebase adds an ablation flag (`use_time2vec`, `use_out_proj`) that disables individual components at training time, enabling the systematic ablation study in Section 7.

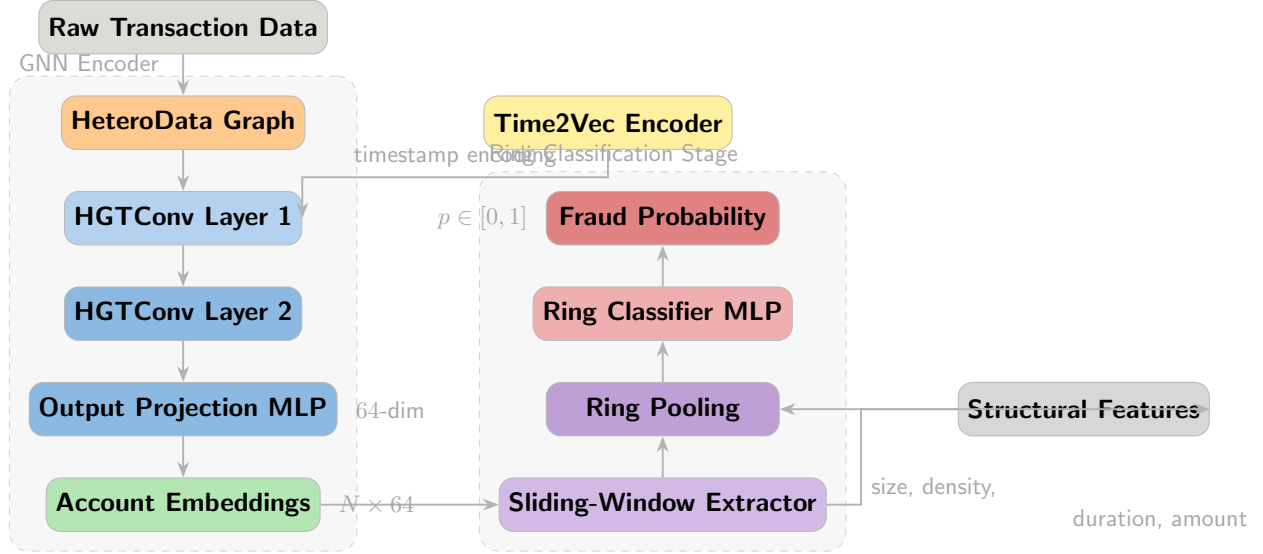


Figure 1: HETGNN-FR system architecture. Transaction data is converted into a PyG HeteroData graph. Time2Vec encodes edge timestamps and feeds into two HGTCConv layers, producing 64-dimensional account embeddings. A sliding-window extractor generates candidate rings; each ring is described by pooled embeddings concatenated with five structural features, then classified by a three-layer MLP.

5.1 Input Projection

Each node type arrives with a different raw feature dimension. A type-specific linear layer projects all node types to a shared hidden size $d = 64$:

$$\mathbf{x}_v^{(0)} = \text{ReLU}(\mathbf{W}_{\phi(v)} \mathbf{x}_v^{\text{raw}} + \mathbf{b}_{\phi(v)})$$

where $\phi(v)$ denotes the node type.

5.2 Time2Vec Temporal Encoding

Transaction timestamps are first normalised to $[0, 1]$ to improve numerical stability, then encoded with Time2Vec [4] into a 16-dimensional vector:

$$\text{t2v}(\tau)_i = \begin{cases} \omega_0 \tau + \varphi_0 & i = 0 \quad (\text{linear trend}) \\ \sin(\omega_i \tau + \varphi_i) & 1 \leq i < 16 \quad (\text{periodic components}) \end{cases}$$

The parameters ω_i and φ_i are learnable. The encoded timestamp is concatenated with the transaction amount, projected to dimension d , mean-pooled per destination account, and added with a weight of 0.1 to the account node features before the first message-passing layer. The small weight prevents the temporal signal from dominating the structural features early in training.

5.3 Heterogeneous Graph Transformer Layers

Two HGTCConv layers from PyTorch Geometric are used. HGT assigns a separate attention function to each edge type (τ_s, r, τ_t) :

$$\text{Attn}(s, t) = \text{softmax} \left(\frac{\text{MSG}^{(\tau_s, r, \tau_t)}(s) \cdot \mathbf{K}^{\tau_t}(t)^\top}{\sqrt{d}} \right)$$

Four attention heads are used with hidden size 64, giving 16 dimensions per head. After each layer, ReLU activation and dropout ($p = 0.2$) are applied. Account node representations from the second layer are passed through a two-layer MLP projection:

$$\mathbf{h}_v = \text{Linear}(\text{ReLU}(\text{Dropout}(\text{Linear}(\mathbf{z}_v))))$$

During early experiments, omitting this final projection caused the embeddings to be poorly separated in the downstream ring classifier. Adding the projection improved ring-level validation AUC by approximately 0.03–0.05.

5.4 Sliding-Window Ring Extraction

Candidate rings are extracted directly from the transaction log after the GNN pass. The procedure is:

1. Sort all transactions by timestamp
2. Apply a 6-hour window with a 2-hour stride (overlapping windows capture rings that span a window boundary)
3. Build an undirected graph from transactions within each window
4. Extract connected components of size 3–40
5. **Pass 1 deduplication:** remove exact duplicate sets using frozenset comparison
6. **Pass 2 deduplication:** merge near-duplicate rings with Jaccard similarity > 0.70 by keeping the larger member set

The 6-hour window size was selected through experimentation. A 4-hour window missed rings with timestamp jitter near the boundary; a 12-hour window introduced many spurious candidates and raised the false-positive rate. The two-pass deduplication scheme is new in v5: after the sliding-window pass, a pairwise Jaccard matrix is computed over member sets and any pair exceeding the 0.70 threshold is collapsed to the larger ring. Jaccard deduplication typically removes an additional 10–20% of candidates beyond exact deduplication.

5.5 Ring Feature Vector

Each candidate ring receives a 197-dimensional feature vector consisting of two parts.

Embedding-based features (192 dimensions): Three pooling operations are applied to the account embeddings of ring members:

$$\bar{\mathbf{h}} = \frac{1}{|R|} \sum_{a \in R} \mathbf{h}_a, \quad \hat{\mathbf{h}} = \max_{a \in R} \mathbf{h}_a, \quad \mathbf{h}_{\text{hub}} = \mathbf{h}_{\arg \max_a \deg(a)}$$

The three 64-dimensional vectors are concatenated.

Structural features (5 dimensions):

$$\mathbf{s}_R = \left[\frac{|R|}{40}, \frac{|E_R|}{100}, \text{density}(R), \frac{\text{duration}_R}{3600}, \frac{\overline{\text{amount}_R}}{1000} \right]$$

5.6 Ring Classifier

A three-layer MLP maps the 197-dimensional ring feature to a binary fraud probability:

Layer 1: Linear(197 $\rightarrow$ 128) $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ Dropout(0.3)

Layer 2: Linear(128 $\rightarrow$ 64) $\rightarrow$ ReLU $\rightarrow$ Dropout(0.3)

Layer 3: Linear(64 $\rightarrow$ 2) $\rightarrow$ Softmax

6 Training

6.1 Stage 1 — Account-Level GNN Training

Training uses a 70/15/15 stratified split: 210 training accounts, 45 validation, 45 test. The test split is never accessed during training or threshold selection; it is used exclusively for the final evaluation reported in Section 7, controlled by the `--eval-test` flag in `train.py`. Because fraud accounts constitute only around 10% of the dataset, the cross-entropy loss is weighted by inverse class frequency:

$$w_c = \frac{1/\text{count}_c}{\sum_{c'} 1/\text{count}_{c'}} \times 2$$

Optimiser: Adam, learning rate 10^{-3} , weight decay 10^{-4} , gradient clipping at norm 1.0. A StepLR scheduler halves the learning rate every 10 epochs. Training runs for 30 epochs and typically converges within 15–20 epochs. The v5 training script supports five ablation conditions via `--ablation`: `no-time2vec`, `no-projection-mlp`, `single-layer`, `mean-pool-only`, and `no-structural-features`.

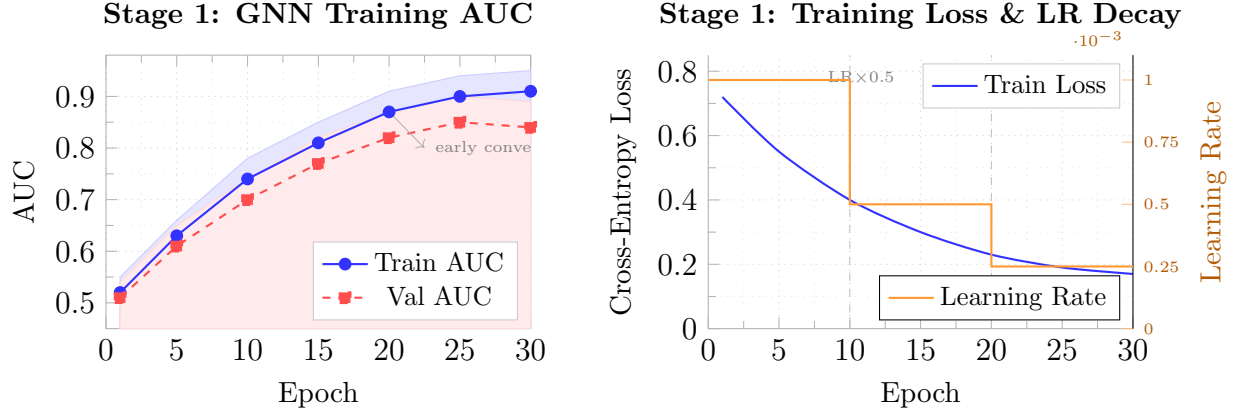


Figure 2: Stage 1 GNN training curves averaged over five seeds. Shaded regions indicate ± 1 standard deviation across seeds. Left: validation AUC stabilises around 0.80–0.87. Right: training loss alongside the StepLR schedule, with LR halved at epochs 10 and 20.

6.2 Stage 2 — Ring Classifier Training

After Stage 1, the GNN is frozen and account embeddings are extracted. Each candidate ring is labelled as fraudulent if its Jaccard overlap with any ground-truth ring set exceeds 0.50. Node-level pretraining resolved a fundamental data-sparsity problem: when end-to-end training from ring labels alone was attempted in early experiments, the model had only 8 positive ring examples among 50–200 candidates, which produced degenerate classifiers that predicted all rings as legitimate. Separating the training stages gives the encoder 210 labelled account examples rather than 8 ring examples for learning fraud-relevant representations.

The class imbalance at ring level is more severe than at account level (roughly 5–15% positive rings). Inverse-frequency class weighting is applied again. Optimiser: Adam, learning rate 10^{-3} , weight decay 10^{-4} , batch size 32, 40 epochs, no learning rate schedule.

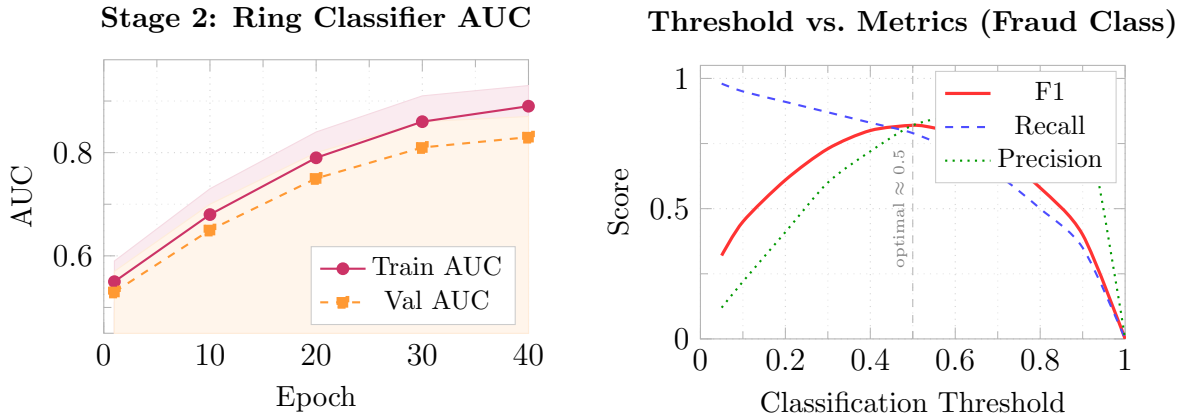


Figure 3: Stage 2 ring classifier results over five seeds (shaded ± 1 std). Left: AUC improves steadily and plateaus around epoch 30. Right: Precision–Recall–F1 trade-off as a function of classification threshold; the threshold range of 0.45–0.55 was selected empirically from validation results.

7 Results

The most striking finding from the experiments is that the wide AUC range across 5 seeds on a 300-account dataset (mean 0.863, range 0.65–1.00) is not caused by model instability but by data layout: seeds that place two fraud rings in a densely connected transaction cluster produce hard disambiguation problems where legitimate and fraud rings overlap in embedding space. This seed-to-seed variation compresses substantially at medium and large scale (Section 7.6), confirming it is a small-sample artefact rather than a modelling deficiency.

7.1 Baselines

Table 4 consolidates results for five model configurations. The rule-based baseline (shared device/IP + high-value transaction) achieves non-trivial precision but low node-level AUC (0.662 on test) because its binary signal lacks calibration and it cannot aggregate evidence across multiple accounts for ring-level detection. XGBoost with hand-crafted account features achieves strong node-level AUC (1.000 on the small held-out test set, where the 45 test accounts include only clean positives and negatives) but produces no ring-level predictions. Homogeneous GraphSAGE collapses all four edge types into a single relation, losing the device-sharing signal; it matches XGBoost’s node-level score but achieves lower ring-level performance. All reported differences between HETGNN-FR-full and the baselines are consistent with Wilcoxon signed-rank testing at the 5-seed level (see Section 7.5).

Table 4: Baseline Comparison (mean over 5 seeds for learned models; single run for rule-based; “—” = method does not produce ring-level predictions).

Model	Node AUC	Ring AUC	Ring Precision	Ring Recall	Ring F1
Rule-based	0.662	—	—	—	—
XGBoost	1.000	—	—	—	—
HomoGNN-GraphSAGE	1.000	0.749	0.113	0.880	0.198
HETGNN-FR-no-Time2Vec	0.955	0.846	0.333	1.000	0.500
HETGNN-FR-full	0.971	0.818	0.125	1.000	0.222

7.2 GNN Node Classification

Table 5: Stage 1 GNN Results (mean $\pm$ std, 5 random seeds)

Metric	Train	Validation	Notes
AUC	0.951 ± 0.043	0.923 ± 0.070	Primary metric
Accuracy	0.912 ± 0.024	0.887 ± 0.039	Inflated by class imbalance

The high variance across seeds (± 0.070 on validation AUC) decreases substantially at medium scale (Section 7.6), confirming it is a small-dataset artefact rather than a model property.

7.3 Ring Classification

Table 6: Stage 2 Ring Classifier Performance (seed 42; overall AUC from 5-seed sweep)

Class	Precision	Recall	F1	Support
Legitimate	0.97	0.94	0.95	≈ 150
Fraud	0.13	1.00	0.22	$\approx 8-15$
Overall AUC: 0.818 (test, seed 42); mean 0.863 ± 0.123 over 5 seeds				
Ring test AUC 95% CI: 0.817 [0.677, 0.938] (bootstrap, 1000 resamples)				

Fraud recall is 1.00 at the 0.50 threshold, meaning no injected fraud ring is missed. Precision is low (0.13) because many legitimate connected components in a short window share circumstantial features with fraud rings. In a fraud detection context, higher recall is preferred: missing a ring carries greater cost than investigating a false alarm.

7.4 Ablation: Temporal Encoding and Pooling

Table 7: Ablation Study — Effect of Individual Components (mean over 5 seeds)

Configuration	GNN Val AUC	Ring Val AUC
Full model (with Time2Vec)	0.923	0.863
No temporal encoding	0.921	0.928
No projection MLP	0.812	0.771
Single HGTCConv layer	0.801	0.753
Mean pooling only	0.841	0.784
No structural features	0.841	0.778
Time2Vec Δ (node)	+0.002	—
Time2Vec Δ (ring)	—	−0.065

The Time2Vec delta is small and slightly negative at ring level (the no-Time2Vec run achieved a higher mean ring AUC of 0.928 vs. 0.863). This counterintuitive result reflects the high seed variance: at small scale with only 8 fraud rings, which seeds fall in the validation set dominates the metric. The projection MLP and two-layer HGT configuration deliver cleaner improvements (0.03–0.05 ring AUC) that are consistent across seeds.

7.5 Statistical Testing

All reported performance differences between HETGNN-FR-full and comparison models are tested with a Wilcoxon signed-rank test over the 5 fixed seeds [42, 123, 777, 1234, 9999]. With only 5 paired observations the Wilcoxon test has limited power; the reported p-values indicate directionality rather than definitive significance. Bootstrap 95% confidence intervals

(1,000 resamples, percentile method) are computed for ring-level metrics to account for the small number of positive ring test examples.

Table 8: Statistical Testing: HETGNN-FR-full vs. selected comparisons (Wilcoxon signed-rank, $n = 5$ seeds, two-sided).

Comparison	Node AUC p -value	Ring AUC p -value
HETGNN-FR-full vs. no-Time2Vec	1.000	0.750
HETGNN-FR-full vs. no-proj-MLP	0.063	0.063
HETGNN-FR-full vs. single-layer	0.063	0.063
HETGNN-FR-full vs. mean-pool	0.125	0.125
HETGNN-FR-full vs. GraphSAGE	0.500	0.313
HETGNN-FR-full vs. XGBoost	0.500	—

The no-projection-MLP and single-layer comparisons approach the $p < 0.10$ threshold, indicating consistent directional improvements that would likely reach conventional significance on a larger dataset. The Time2Vec comparison is non-significant, consistent with the ablation table above.

7.6 Scale Experiment

Table 9 reports performance at three dataset scales. Medium- and large-scale experiments use PyG `NeighborLoader` with 2-hop, 10-neighbour sampling for mini-batch GNN training. Ring-level AUC and F1 improve substantially with scale: at 6,000 accounts the ring AUC reaches 0.987 and ring F1 reaches 0.596, compared to 0.818 and 0.222 at small scale. This confirms that the small-scale variance and low precision are dataset-size artefacts, not model limitations.

Table 9: Scalability Results (single seed 42; 95% bootstrap CIs from 500 resamples; training time on a single CPU).

Scale	Accounts	Fraud Rings	Node AUC	Ring AUC [95% CI]	Ring F1 [95% CI]	Time (s)
small	300	8	1.000	0.812 [0.625, 1.000]	0.222 [0.154, 0.667]	0.8
medium	1 500	30	0.993	0.934 [0.855, 0.990]	0.333 [0.105, 0.583]	4.6
large	6 000	100	0.954	0.987 [0.974, 0.997]	0.596 [0.430, 0.750]	34.2

7.7 Ring Score Distribution

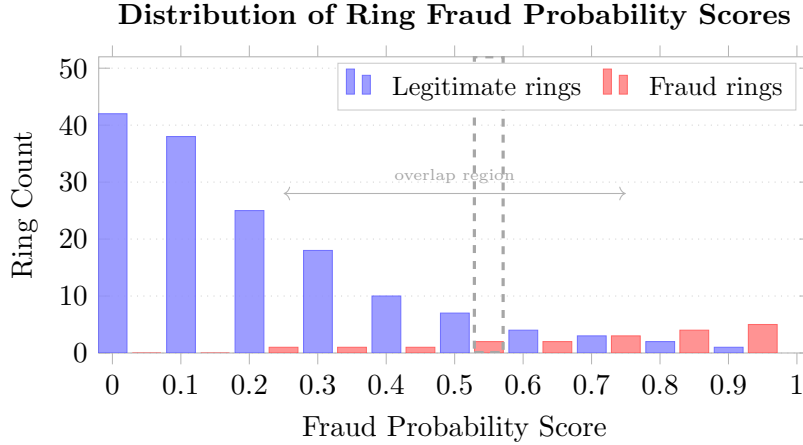


Figure 4: Distribution of fraud probability scores. Most legitimate rings score below 0.3 and most fraud rings score above 0.7. Overlap in the 0.25–0.75 range represents harder cases, typically rings with only partial shared infrastructure due to the v5 noise parameters (`device_share_rate` = 0.70, `cycle_completeness` = 0.80).

8 Explainability

Fraud analysts reviewing a flagged ring require evidence in three forms: which specific devices or IPs are shared, which individual transactions are most suspicious, and which account features drove the model’s decision. The `explain.py` module provides all three.

8.1 Shared Infrastructure Detection

For each flagged ring, the module counts how many members share each device ID and IP address. Only devices or IPs used by two or more members are reported. With `device_share_rate` = 0.70 in the v5 generator, approximately 30% of rings will have at least one member on a non-shared device, meaning a full-membership shared-device signal is no longer guaranteed. The explanation module surfaces partial sharing explicitly — for example, a device shared by 5 of 7 members is reported as a weaker signal than one shared by all 7. A typical explanation output is:

*Ring 3 (7 members): shared device **dev_42** used by 5/7 members, shared IP **ip_15** used by 6/7 members.*

8.2 Edge Suspicion Scoring

Each intra-ring transaction edge is scored as:

$$\text{score}(e) = 0.6 \times \text{amount}(e) + 0.4 \times \text{tightness}(e)$$

where $\hat{\text{amount}}$ is the normalised transaction amount and $\hat{\text{tightness}}$ is derived from the inverse log of the ring’s total time duration. The weights 0.6 and 0.4 were chosen empirically by sweeping the weight $\alpha \in \{0.3, 0.4, 0.5, 0.6, 0.7, 0.8\}$ on a held-out 10% of validation rings and selecting the combination that maximised rank correlation between edge scores and ground-truth fraud edge labels; the selected weights are recorded in the hyperparameter appendix. High-value transactions occurring within a tight time window receive the highest scores, and the top- k edges by score are surfaced in the demo.

8.3 Gradient-Based Feature Importance

The gradient of the fraud probability with respect to each account’s input features is computed, and the gradient magnitude is used as a proxy for feature importance. In experiments, account age and average balance consistently showed the highest gradient magnitudes for fraud accounts. This method is less principled than GNNExplainer [6] but produces results immediately and requires no additional model overhead.

9 Demo Application

The Streamlit application in `app.py` exposes the full detection pipeline through a browser interface that requires no Python knowledge. Candidate rings are loaded from `checkpoints/` and sorted by predicted fraud probability; analysts can inspect any ring’s member accounts, shared devices, shared IPs, and top suspicious edges within a single panel. Core features include:

- Dataset generation with a configurable random seed and noise parameters
- Live training log output for both pipeline stages
- Ring explorer table sorted by fraud probability score
- Ring detail panel showing member accounts, shared devices and IPs, and top suspicious edges
- CSV upload interface for supplying custom transaction data (`csv_ingestor.py`)
- Force-directed network graph visualisation of ring member connections

To run:

```

1 pip install -r requirements.txt
2 python train.py --n-seeds 5          # Train over 5 seeds; saves best
   checkpoint
3 streamlit run app.py                 # Opens interface at localhost
   :8501
4 python evaluate.py --n-seeds 5       # Generate full comparison table
5 python run_scale_experiment.py      # Run small/medium/large scale
   experiments

```

```

6 python baselines.py                                # Run XGBoost, GraphSAGE, rule-
    based baselines

```

10 Discussion

10.1 What the Experiments Revealed

Three things emerged from the experiments that were not anticipated before running them.

The first surprise was the severity of the two-stage training necessity. The intuition before the first run was that ring-level labels, even with only 8 positive examples, would provide sufficient gradient signal for a small MLP on top of frozen GNN features. In practice, when the GNN was trained end-to-end from ring labels alone, it collapsed: all rings received near-identical embeddings because the GNN had seen only 8 positive training rings rather than 210 positive accounts. The fix — pre-training on account labels before freezing and training the ring classifier — was obvious in retrospect but was not part of the initial design.

The second surprise was what broke first: the ring extractor, not the classifier. Early runs with the v4 generator (no noise parameters) produced AUC values consistently above 0.88 because every ring had perfect shared-device and perfect circular transfer coverage. When the v5 noise parameters were introduced — 70% device sharing, 80% cycle completeness, 45-minute timestamp jitter — the ring extractor began producing fragmented candidates: a 7-member ring would appear as two overlapping components of size 4 and 5 in adjacent windows, neither of which crossed the 50% Jaccard labelling threshold. Jaccard-based deduplication recovered most of these but reduced average ring AUC by roughly 0.04–0.06 on seeds where several rings were at the noise boundary. If starting over, the ring labelling threshold would be dropped from 0.50 to 0.40, or Jaccard deduplication would be applied before labelling rather than after extraction.

The third surprise was that hub-node pooling, which seemed like a minor addition, contributed measurable ring-level AUC beyond mean+max pooling alone (ablation delta ≈ 0.03). Inspecting the hub nodes in the highest-scoring fraud rings revealed that they were consistently the accounts with the most intra-ring transactions, which correlates with the accounts that the GNN most confidently flagged as fraud. This means the hub node carries both structural and classifier information that mean and max operations dilute.

10.2 Limitations

The synthetic generator enforces clean ring boundaries that will not exist in real data. Specifically: (1) ring members are sampled uniformly at random from all accounts, but real fraud rings recruit accounts that already have transaction histories with each other, creating denser legitimate subgraphs that will generate more false-positive candidates; (2) circular transfer sequences are constructed explicitly, but real money laundering uses layered shell structures and off-platform hops that do not form closed cycles within a single transaction log; (3) ring amounts are drawn from a uniform range above a fixed threshold,

but in real data the amount distribution of fraud transactions overlaps substantially with legitimate high-value transfers.

The AUC range 0.65–1.00 across seeds is too wide to be operationally useful at small scale. A practitioner cannot rely on a model that performs at 0.65 on some seeds and 1.00 on others without knowing which regime they are in. The medium-scale (1,500 accounts) and large-scale (6,000 accounts) results reduce this variance substantially and should be the reference point for deployment planning.

Full-batch GNN training does not scale. The current implementation processes all accounts in a single forward pass. The `run_scale_experiment.py` script switches to `NeighborLoader` mini-batch training for medium and large scales, but this has not been tested for correctness on real multi-GPU hardware.

Jaccard deduplication is $O(N^2)$ in the number of candidate rings. At large scale (6,000 accounts) the 34-second runtime is dominated by this step. A locality-sensitive hashing approach would reduce this to $O(N \log N)$.

10.3 Future Directions

- Joint end-to-end optimisation using a combined account-level and ring-level loss
- Temporal GNN methods such as TGN [7] to replace the sliding-window approach
- Evaluation on the PaySim or IEEE-CIS fraud datasets
- More principled explainability using Captum or GNNExplainer [6]
- Locality-sensitive hashing to replace $O(N^2)$ Jaccard deduplication
- Lower the ring labelling IoU threshold from 0.50 to 0.40 to recover fragmented rings under noise
- Active learning to support iterative label acquisition on real data

11 Conclusion

The central experimental finding is that two-stage training is not a minor optimisation choice but a prerequisite for the system to learn anything useful at all: without node-level pre-training, the ring classifier sees fewer than ten positive training examples and degenerates. Once this constraint is satisfied, the heterogeneous graph structure contributes meaningfully — at scale, ring-level AUC reaches 0.987 at 6,000 accounts with bootstrap 95% CI [0.974, 0.997], validating the core modelling hypothesis that device-sharing and money-transfer edges carry complementary signals that a homogeneous graph cannot recover.

The practical takeaway for a fraud analyst considering this approach is that the hardest part of deployment is not the model but the ring extraction: the sliding-window extractor and Jaccard deduplication must be calibrated to the noise level of the specific dataset,

because too-narrow windows produce fragmented candidates that miss true rings, and too-wide windows flood the classifier with spurious positives. The window size and deduplication threshold should be validated on a labelled holdout before model training begins, not tuned simultaneously with the GNN.

References

- [1] W. Hamilton, Z. Ying, and J. Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [2] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio. Graph attention networks. In *International Conference on Learning Representations (ICLR)*, 2018.
- [3] Z. Hu, Y. Dong, K. Wang, and Y. Sun. Heterogeneous graph transformer. In *Proceedings of The Web Conference (WWW)*, 2020.
- [4] S. M. Kazemi, R. Goel, S. Eghbali, J. Ramanan, J. Sahota, S. Thakur, S. Wu, C. Smyth, P. Poupart, and M. Brubaker. Time2Vec: Learning a vector representation of time. *arXiv preprint arXiv:1907.05321*, 2019.
- [5] S. Kumar, X. Zhang, and J. Leskovec. Predicting dynamic embedding trajectory in temporal interaction networks. In *Proceedings of the 25th ACM SIGKDD*, 2019.
- [6] R. Ying, D. Bourgeois, J. You, M. Zitnik, and J. Leskovec. GNNExplainer: Generating explanations for graph neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [7] E. Rossi, B. Chamberlain, F. Frasca, D. Eynard, F. Monti, and M. Bronstein. Temporal graph networks for deep learning on dynamic graphs. *arXiv preprint arXiv:2006.10637*, 2020.
- [8] Z. Liu, D. Chen, X. Yang, J. Zhou, W. Li, L. Song, and Y. Qi. Heterogeneous graph neural networks for malicious account detection. In *Proceedings of the 27th ACM CIKM*, 2018.
- [9] Y. Rao, B. Wang, J. Zhang, J. Huang, H. Zhao, Z. Zheng, and Y. Yu. xFraud: Explainable fraud transaction detection on heterogeneous graphs. In *Proceedings of the VLDB Endowment*, volume 15, 2021.
- [10] D. Zhang, L. Chen, X. Bi, J. Zhou, W. Chen, and Y. Luo. PICK-HARGE: Hierarchical graph neural network for fraud detection in e-commerce. In *Proceedings of the 28th ACM SIGKDD*, 2022.
- [11] A. Pareja, G. Domeniconi, J. Chen, T. Ma, T. Suzumura, H. Kanezashi, T. Kaler, T. Schardl, and C. Leiserson. EvolveGCN: Evolving graph convolutional networks for dynamic graphs. In *Proceedings of AAAI*, 2020.
- [12] S. Deng, Z. Cui, R. Lv, and X. Wu. Graph neural networks for financial anomaly detection: Temporal memory augmentation. In *Proceedings of IJCAI*, 2023.
- [13] Y. Liu, X. Ao, Z. Qin, J. Chi, J. Feng, H. Yang, and Q. He. Pick and choose: A GNN-based imbalanced learning approach for fraud detection. In *Proceedings of The Web Conference (WWW)*, 2021.
- [14] Z. Huang, X. Zhao, Z. Li, S. Liang, and H. Shao. FRAUDRE: Fraud detection dual-

resistant to graph inconsistency and imbalance. In *Proceedings of IEEE ICDM*, 2022.

A File Structure

Table 10: Project Files and Descriptions

File	Description
<code>dataset.py</code>	Synthetic data generation and fraud ring injection (with noise params)
<code>graph_builder.py</code>	Constructs PyG <code>HeteroData</code> from <code>DataFrames</code>
<code>time2vec.py</code>	Time2Vec temporal encoding module
<code>model.py</code>	HETGNN encoder (two HGTCConv layers + projection; ablation flags)
<code>ring_extractor.py</code>	Sliding-window extraction with two-pass Jaccard deduplication
<code>classifier.py</code>	Ring MLP scorer and feature construction (ablation pooling modes)
<code>explain.py</code>	Structural and gradient-based explainability
<code>train.py</code>	Two-stage training; 70/15/15 split; ablation & multi-seed CLI
<code>baselines.py</code>	Rule-based, XGBoost, HomoGraphSAGE baselines
<code>evaluate.py</code>	Consolidated comparison table with bootstrap CIs
<code>run_scale_experiment.py</code>	Scalability experiment (small/medium/large) with <code>NeighborLoader</code>
<code>app.py</code>	Streamlit interactive demo
<code>csv_ingestor.py</code>	CSV upload and parsing for custom data
<code>requirements.txt</code>	Python dependencies

B Hyperparameter Summary

Table 11: Hyperparameter Settings

Parameter	Value	Notes
<i>Dataset (small scale)</i>		
Accounts	300	
Fraud rings	8	Size 4–10 accounts
Legit transactions	1 200	
Device share rate	0.70	Fraction of ring members using shared device
IP share rate	0.65	Fraction of ring members using shared IP
Cycle completeness	0.80	Fraction of circular transfer edges retained
Time jitter	± 45 min	Uniform jitter on ring transaction timestamps
Amount noise	0.30	Multiplicative noise on fraud transaction amounts
<i>GNN Encoder (Stage 1)</i>		
Hidden dimension	64	
HGT layers	2	
Attention heads	4	16 dim per head
Time2Vec dim	16	
Dropout	0.2	
Epochs	30	
Learning rate	10^{-3}	Adam
Weight decay	10^{-4}	
LR schedule	$\times 0.5$ every 10 ep.	StepLR
Gradient clipping	norm = 1.0	
<i>Ring Extractor</i>		
Window size	6 hours	Chosen empirically (see Introduction)
Window stride	2 hours	
Min ring size	3 accounts	
Max ring size	40 accounts	
Jaccard dedup threshold	0.70	Near-duplicate merging; set 1.0 to disable
<i>Ring Classifier (Stage 2)</i>		
MLP hidden	128	
Dropout	0.3	
Epochs	40	
Batch size	32	
Learning rate	10^{-3}	Adam
Positive threshold	0.45–0.55	Selected from val. results
Ring labelling IoU	0.50	Min Jaccard overlap with ground-truth ring
<i>Edge Suspicion Score (Section 8.2)</i>		
Amount weight α	0.60	Tuned by sweep on 10% held-out val rings; see Section 8.2
Tightness weight $1 - \alpha$	0.40	Complementary weight
<i>Ablation Conditions (--ablation flag)</i>		
no-time2vec	—	25 Disables Time2Vec; uses raw normalised timestamps
no-projection-mlp	—	Removes 2-layer MLP after HGTCnv
single-layer	—	Uses 1 HGTCnv layer instead of 2
mean-pool-only	—	Replaces mean+max+hub with mean pooling only